

Enums and matching

An `enum` lists the shapes a value is allowed to take. Each shape may carry data of its own.

```
use std.io

enum Shape {
    Circle(Num)
    Rect(Num, Num)
    Empty
}

fn area(s: Shape) -> Num {
    give match s {
        when Circle(r) -> 3.14159 * r * r
        when Rect(w, h) -> w * h
        when Empty -> 0.0
    }
}

fn main() {
    io.say("{area(Circle(2.0))}")
    io.say("{area(Rect(3.0, 4.0))}")
    io.say("{area(Empty)}")
}
```

`match` pulls the data out as it chooses the arm. The compiler checks that every shape is covered, and names the ones you forgot.

Matching on anything

`match` is not only for enums.

```
use std.io

fn describe(n: Int) -> Text {
    give match n {
        when 0 -> "nothing"
        when 1 -> "one"
        when 2 -> "a pair"
        else -> if n < 0 { "below zero" } else { "many" }
    }
}

fn main() {
    for n in [-3, 0, 1, 2, 17] {
        io.say("{n} is {describe(n)}")
    }
}
```

Guards

`if` after a pattern adds a condition.

```
use std.io

enum Reading { Temperature(Num) Pressure(Num) }

fn comment(r: Reading) -> Text {
  give match r {
    when Temperature(t) if t > 38.0 -> "fever"
    when Temperature(t) -> "temperature {t} is fine"
    when Pressure(p) if p < 90.0 -> "pressure is low"
    when Pressure(p) -> "pressure {p} is fine"
  }
}

fn main() {
  io.say(comment(Temperature(39.4)))
  io.say(comment(Pressure(120.0)))
}
```

List patterns

```
use std.io

fn summarise(items: [Int]) -> Text {
  give match items {
    when [] -> "empty"
    when [one] -> "just {one}"
    when [first, second] -> "{first} and {second}"
    when [first, ..rest] -> "{first} then {rest.len()} more"
    else -> "something else"
  }
}

fn main() {
  io.say(summarise([]))
  io.say(summarise([5]))
  io.say(summarise([1, 2]))
  io.say(summarise([1, 2, 3, 4]))
}
```

Exercise

Model a simple calculator: an enum with `Add`, `Subtract`, `Multiply` and `Divide`, each carrying two numbers, and a function that works out the answer.